nature reviews methods primers

https://doi.org/10.1038/s43586-024-00294-7

Primer

Check for updates

Graph neural networks

Gabriele Corso[1,3] ✉, Hannes Stark[1,3] ✉, Stefanie Jegelka[1,2], Tommi Jaakkola[1] & Regina Barzilay[1] ✉

Abstract

Graphs are flexible mathematical objects that can represent many entities and knowledge from different domains, including in the life sciences. Graph neural networks (GNNs) are mathematical models that can learn functions over graphs and are a leading approach for building predictive models on graph-structured data. This combination has enabled GNNs to advance the state of the art in many disciplines, from discovering new antibiotics and identifying drug-repurposing candidates to modelling physical systems and generating new molecules. This Primer provides a practical and accessible introduction to GNNs, describing their properties and applications to the life and physical sciences. Emphasis is placed on the practical implications of key theoretical limitations, new ideas to solve these challenges and important considerations when using GNNs on a new task.

Sections

Introduction

Experimentation

Results

Applications

Reproducibility and data deposition

Limitations and optimizations

Outlook

[1]CSAIL, MIT, Cambridge, MA, US. [2]School of CIT, TU Munich, Munich, Germany. [3]These authors contributed equally: Gabriele Corso, Hannes Stark. ✉e-mail: gcorso@mit.edu; hstark@mit.edu; regina@csail.mit.edu

Introduction

A wide variety of problems can be modelled as graph structures. These mathematical objects consist of nodes, which represent entities, and edges, which capture their relationships or interactions. Graphs can represent a diverse set of data types, from molecules, in which nodes are atoms and bonds are edges, to biomedical knowledge graphs, which link hundreds of thousands of genes, drugs and diseases.

Until recently, inference on graph structures was only possible through a fixed set of rules or features designed for each prediction task. For example, molecules were represented by a vector whose entries capture specific substructures and patterns known to be chemically relevant. Although such approaches can reflect valuable domain knowledge, their representation power is limited to what is known. They lack the flexibility to capture novel, complex patterns that may be important for problems of interest.

To overcome this challenge, a new class of deep learning methods, referred to as graph neural networks (GNNs), has emerged[1–3], and it has been successfully applied to many problems in the life sciences and beyond. Since their first introduction, the standard formulation of GNNs has evolved, and this Primer presents their modern view. Similarly to other deep learning architectures, GNNs use available training data for the problem to extract representations from graphs. These learned representations are high-dimensional vectors that encode task-relevant information in a machine-understandable format. For instance, a GNN can be trained on molecules to induce representations containing relevant information about their toxicity.

The GNN modelling framework can answer questions about a graph's nodes, edges or the full graph by formulating problems as prediction tasks. For example, inferring a molecule's toxicity is a graph-level prediction. At the same time, given a gene regulatory network, information may be desired about the function of an orphan gene (node prediction) or previously unknown interactions (edge prediction).

Despite their wide applicability, GNNs are not without limitations. For instance, if the labelled training examples fail to capture the diversity or breadth of the intended deployment data, learned models generalize poorly to real-world scenarios. Furthermore, interpretability and uncertainty estimation remain a challenge for large models. Finally, many GNN architectures lack the ability to recognize certain critical patterns in graphs. Therefore, although GNNs have proven successful in many applications, it is essential to understand their inner workings, strengths and weaknesses to maximize their effectiveness. This Primer aims to introduce these components, alongside an exploration of solutions and ongoing research to address the shortcomings of the technique.

In the following sections, the GNN framework, applications, limitations and directions for future research are introduced. The Experimentation section defines the GNN framework, using a molecular property prediction example to illustrate practical strengths and weaknesses. The Results section elaborates on different variations, theoretical properties and shortcomings of GNNs, as well as some proposed solutions. The Applications section features several successes of GNNs, with a focus on fundamental modelling principles, benchmark datasets and best practices. Finally, the Limitations section highlights some of the fundamental limitations of GNNs, ranging from data dependency to the technical limits of the mathematical frameworks. The article concludes with an outlook of pressing issues and promising directions for GNNs.

This Primer is intended as an introduction to GNNs for students and practitioners. It is assumed that readers are familiar with basic concepts in machine learning, including featurization, gradient descent and train/test splits. This is not a comprehensive survey of all the theoretical results and methodologies regarding GNNs. Interested readers are directed to refs. 4,5 for more exhaustive expositions on the theoretical foundations.

Experimentation

This section introduces the fundamental building blocks of GNNs and some key practical considerations by working through a motivating example: predicting whether a small molecule inhibits HIV growth.

Molecular property predictors have become a fundamental tool in biochemistry, in which in silico virtual screening offers a promising alternative to expensive assays. Classical methods for molecular property prediction begin with a molecular fingerprint, a vector of hand-crafted features based on the molecular graph[6]. These features can be used as an input in simple machine learning models — for example, random forests, support vector machines and shallow feedforward networks — for property prediction. However, these handcrafted features are limited to existing knowledge about molecules.

To move beyond molecular fingerprints, GNNs were developed to learn more expressive and powerful representations directly from the graphs. Given a molecular graph as input, a GNN can be trained to predict properties such as drug absorption, distribution, metabolism, excretion, toxicity, protein binding affinity and solubility (Fig. 1). The implementation of the method accompanying this example is available in a Jupyter notebook on the GitHub repository provided.

Formalization

In the worked example, the molecule is viewed as a graph. A graph is a tuple $G = (V, E)$ in which V is the set of nodes v_i — for instance, the atoms — and E is the set of edges (v_i, v_j) connecting pairs of nodes, in this case, the covalent bonds between atoms. The set of neighbours of a node v_i are denoted as $N_i = \{v_j | (v_i, v_j) \in E\}$.

The nodes v_i are first converted to machine-understandable vector representation, $h_i^{(0)}$, in which the exponent 0 indicates the input, before the first layer. For instance, each atom type — for example, hydrogen, carbon or oxygen — is associated with a specific high-dimensional embedding. Similarly, edges can have representations e_{ij} that encode their properties, such as the bond type (single or double) between two atoms.

Message-passing

The fundamental building block of a GNN is the message-passing layer. At every message-passing layer, each node collects messages in the form of vector representations from its neighbouring nodes, aggregates them into a single vector, and uses this vector to update its own representation. Iterative message-passing enables each node to build representations that capture larger, more complex patterns from the graph around them. The filters learned by each message-passing layer depend on the transformations applied to the messages' representations before and after the aggregation, which are parameterized with feedforward neural networks (FF-NNs) and whose weights are learned from data with stochastic gradient descent.

In the worked example, a simple instantiation of message-passing is used, in which the messages are the representations of the neighbours, the aggregation strategy is the vector summation of all messages, and the update is performed with a one-layer FF-NN with

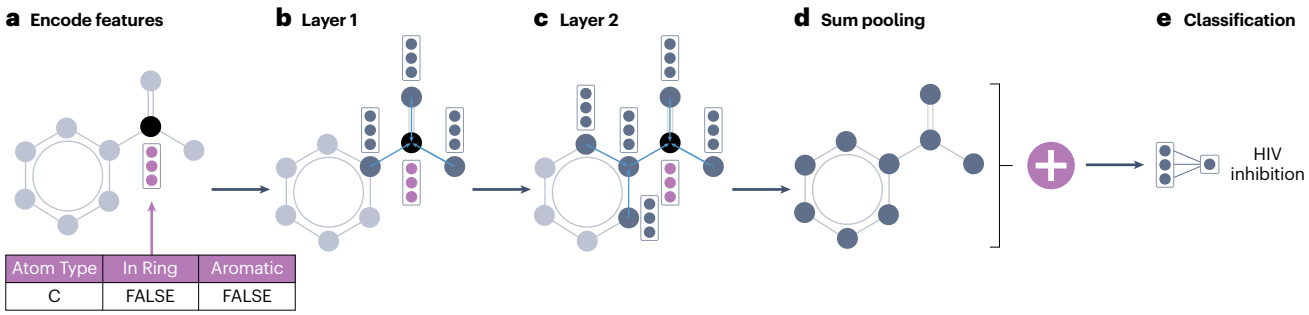

Fig. 1 | Molecular property prediction example: given a molecule, a GNN predicts its ability to inhibit HIV replication. a, A molecule, for example specified by its simplified molecular-input line-entry system string, is converted into a graph, and the node representations are initialized as vectors describing the atom. **b**, In the first message-passing layer, an example node receives messages from its neighbours and updates its representation based on them. Its representation now contains information from a one-hop neighbourhood. **c**, In the second message-passing step, the example node receives a message from one of its neighbours that received messages from more distant nodes in the previous layers. The representation now contains information from a two-hop neighbourhood. **d**, The representations of all nodes are aggregated into a single vector by summing them. **e**, A feedforward neural network takes the produced vector representation and outputs a single logit to classify whether the molecule inhibits HIV replication. GNN, graph neural network.

learnable linear transformation $W^{(l)}$ and ReLU non-linearity. Therefore, the mathematical operation performed to update the representation $h_i^{(l)}$ of node i at layer ℓ can be written as:

$$h_i^{(l+1)} = ReLU(W^{(l+1)}(h_i^{(l)} + \sum_j h_j^{(l)}))$$

Final prediction

After the final message-passing layer, the representations of individual nodes are aggregated and transformed to make task-specific predictions, as different problems may require outputs at different scales.

For example, the molecular property prediction task is a graph-level problem in which to make a single prediction for the graph, the representations of all nodes — whose count may vary across molecules — must be aggregated into a fixed-size vector that represents the whole molecule. In the provided implementation, after four message-passing layers, the final prediction (likelihood of inhibition) is reached by summing the nodes' features in the last layer and passing their sum through a linear layer with output dimension 1.

By contrast, in node-level tasks, such as the functional characterization of proteins in a protein interaction network, the node representations after the message-passing layers can be directly used as outputs for prediction. Finally, the most common class of edge-level tasks is link prediction, in which the model is trained to predict missing edges in the graph, for example knowledge graph completion or recommendation systems. For this, a classifier is typically trained by aggregating the final representations of the two nodes or surrounding subgraphs connected by the edge in question.

Efficient implementation

In many applications, GNNs are run on graphs with thousands or millions of nodes. In such cases, efficient sparse implementations of message-passing are necessary to run training and inference in a reasonable time. The computational complexity of a message-passing layer is $O(|E| + |V|) = O(|E|)$, in which O indicated the big O notation, or linear in the number of edges, as messages have to be computed for every edge and in connected graphs $|E| \geq |V| - 1$. To run these operations efficiently on graphics processing unit or tensor processor unit hardware, it is critical to parallelize message computation and aggregation. One approach to parallelize these operations is to store edge information and messages as pairwise matrices, which can be efficiently transformed via matrix and dot products. However, this method would incur a runtime and memory complexity of $O(|V|^2)$, which is quadratic in the number of nodes, and for large sparse graphs it can be substantially larger than $O(|E|)$.

Instead, if the graph is represented in a sparse matrix data structure or adjacency list, computations can be parallelized while maintaining the $O(|E|)$ complexity. For large structures, such as the atomic resolution graph of a protein (typically 1,000–10,000 atoms) or large knowledge graph (>100,000 nodes[7]), sparse implementations enable the data to fit in memory, meaning the computation can be completed orders of magnitude faster. As these sparse computations require careful implementation, specialized libraries for GNNs have been developed. The most widely used include PyTorch Geometric[8] and Deep Graph Library[9] for general graphs, Chemprop[10] for molecular graphs, and e3nn[11] for 3D geometric graphs. These libraries provide instantiations of existing models, simplify the implementation of novel architectures (see Jupyter notebook on the GitHub repository provided) and give access to datasets and auxiliary tools, such as featurization.

Data format and splitting

When using deep neural networks like GNNs, a key question is whether the features learned from the training data will generalize to real-world scenarios. To tackle this question without collecting additional data, splitting data between training and testing is critical. For graphs, data-splitting approaches differ between inductive and transductive settings.

Inductive tasks. Inductive tasks closely resemble the common paradigm of machine learning problems in which the training, validation and testing datasets involve separate objects. Each set contains different graphs over which the GNNs are trained and evaluated. Molecular property prediction is a common example, as models are trained and tested on different sets of molecules. Deciding how to split the graphs between the different sets often requires domain expertise. In drug discovery, although labelled data are sourced from commonly observed parts of molecular space, to find novel drugs, unexplored parts of the

chemical space are searched. To simulate this distribution shift, the community commonly uses scaffold splits (Fig. 2) or time splits, in which molecules in the training, validation and test sets have different molecular scaffolds or are sourced from experiments conducted over different time periods.

Transductive tasks. Transductive tasks (semi-supervised learning) train and test on the same graph, which is typically large and incomplete. For example, the goal of knowledge graph completion is to detect missing edges based on existing ones. In biological settings, it may be desirable to repurpose existing drugs for new diseases. In this case, drugs and diseases are nodes, whereas efficacy relationships are edges. Care must be taken when dividing the known edges of this single graph into training and testing splits. Randomly masking edges between drugs and diseases may lead the model to just learn that similar drugs are likely to work against similar diseases. Although valid, this conclusion would not allow the discovery of drugs for diseases that lack known treatments.

Training and evaluation

The HIV inhibition prediction example is an inductive task, which uses data provided by the Drug Therapeutics Program of the NIH's National Cancer Institute, accessible from the Open Graph Benchmark (OGB)[12]. The dataset contains 40,000 small molecules, together with a binary label indicating their ability to inhibit HIV growth. The standardized data splits from the OGB use scaffold splitting, with 80% of the molecules for training, 10% for validation and 10% for testing.

The example GNN is constructed based on the previously described architecture, with an embedding layer, four layers of message passing and a final add pooling and feedforward network. For this classification task, cross-entropy is used as the loss function. To evaluate model performance, the area under the receiver operating curve is measured (ROC-AUC).

After training for 100 epochs, the ROC-AUC is 82.5% on the training set compared with 73.0% on the validation set. The higher ROC-AUC for training is expected, as the model sees those scaffolds during its training process. However, such a substantial difference might indicate overfitting. Overfitting means that a model has so many parameters that it is able to memorize the training data and labels instead of learning to recognize patterns that generalize to unseen data points. This is a common problem for machine learning algorithms in data-scarce settings, which is often the case in the life sciences. To ensure that a method is useful for new data, it is crucial to check if overfitting occurred and to evaluate generalization capabilities, for instance, via scaffold splits (Fig. 2).

In the worked example, overfitting can be avoided by stopping the training early. As the losses are tracked across training, the training process can be stopped at the point of highest validation performance (77.9% ROC-AUC) before the model starts overfitting, which translates to a 74.5% ROC-AUC on the test set. This is considerably better than the performance (70.5% test set ROC-AUC) obtained with a shallow FF-NN on Morgan fingerprints.

Results

Properties of GNNs

Although deep learning models offer a way to learn complex patterns directly from raw data, this usually comes at the cost of data efficiency. Given the large number of parameters to optimize, if the number of labelled examples is not large enough, the deep learning models are likely to learn spurious correlations and miss patterns that would enable generalization to unseen data points. A key to the success of GNNs is that, compared with standard FF-NNs, they improve data efficiency and accuracy on graph-structured data due to two fundamental properties: locality bias and permutation equivariance.

Locality bias. Whenever data are represented as graphs, edges are drawn to connect objects with some relation to one another. It is thus natural to think that a better representation of a node can be built by looking at its neighbours, to provide more information than looking at another node at random. This locality inductive bias is the basis of the message-passing concept and induces the model to learn more generalizable functions[4].

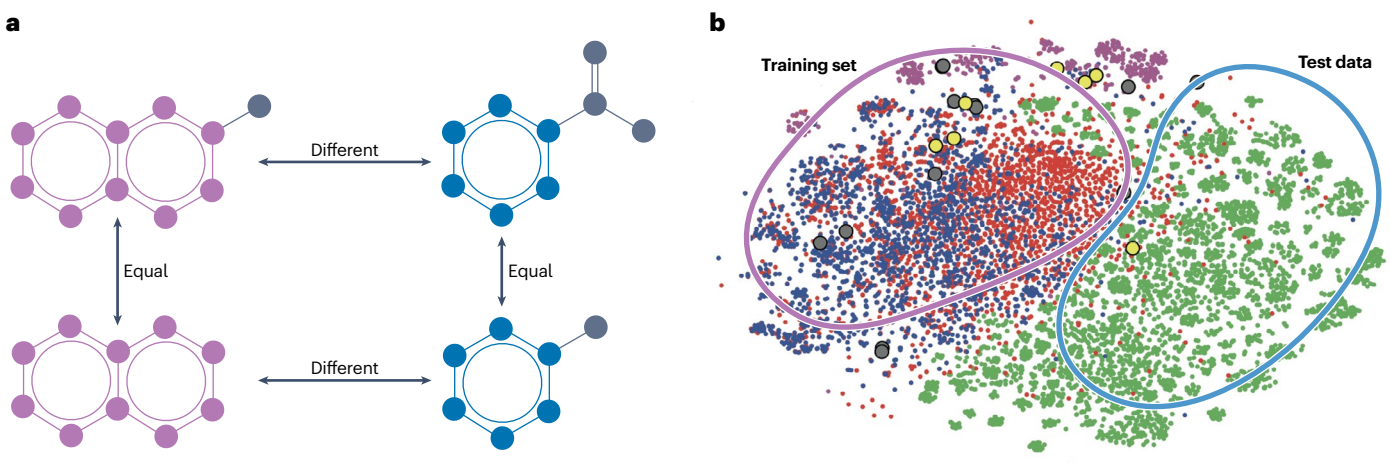

Fig. 2 | Molecule similarity and overfitting. a, Examples of molecules that have the same or a different molecular scaffold (indicated by purple and blue colour), which is a core substructure. **b**, A clustered 2D embedding of molecules. Each point corresponds to a molecule, and similar ones are clustered together. Points' colouring corresponds to different data sources. The larger yellow points and grey points correspond to true positive and false positive antibiotic activity predictions of a graph neural network. A scaffold split ensures that no molecules in the training data (purple curve) and test data (blue curve) have the same scaffold. The purpose is to evaluate the model's capability to generalize to a test distribution that is substantially different from the training data, which is expected in real-world applications. Part **b** adapted with permission from ref. 67, Elsevier.

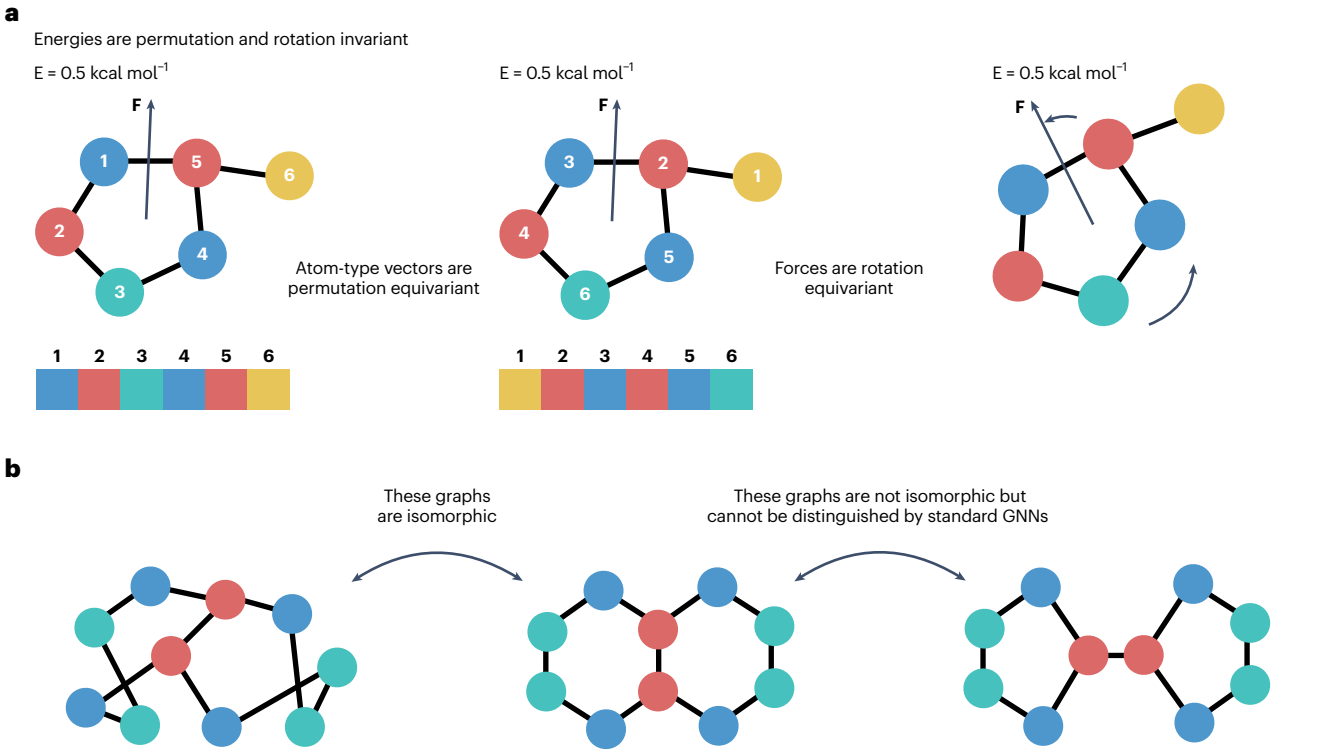

Fig. 3 | Important data symmetries for GNNs. a, Examples of properties that are permutation and rotation invariant or equivariant. The energy of a molecule does not depend on its frame of reference, and so it is translation and rotation invariant. By contrast, the forces are translation and rotation equivariant vectors, as they rotate with the molecule in the new frame of reference. The energy is invariant with respect to the reordering of the atoms, whereas the vector of atom types or charges is rearranged with the same ordering. **b**, The challenge of graph isomorphism: the Weisfeiler–Leman test as a standard graph neural network (GNN) will never be able to distinguish the third graph from the first two, as nodes of the same colour will always have the same representation.

Permutation invariance and equivariance. Alongside soft inductive biases, data efficiency can be improved by building data symmetries into the architecture. This reduces the number of possible functions the model can represent and avoids learning meaningless correlations between specific node orderings and labels, making it more likely that the model will learn a generalizable function. For graphs, the most important symmetry is permutation invariance, according to which the graph under consideration does not change if the nodes are permuted, meaning node ordering does not matter.

This idea is formalized using the group theoretic concepts of invariance and equivariance. First, a set of transformations — formally a group[13] — needs to be defined, such as permuting the order of nodes. A function is considered invariant to this set of transformations if its output does not change when one of the transformations is applied to its input. Similarly, a function is equivariant if informally applying a transformation to the input leads to a corresponding transformation of the output.

Graph-level outputs should be invariant to permutations. For example, a vector representation of a molecule should be the same regardless of the order in which the atoms are written in. On the other hand, node-level predictions are equivariant. For example, the vector containing the predicted electronegativity of each atom should directly depend on the ordering used to represent the atoms in the graph (Fig. 3a). These properties are achieved in GNNs if permutation-invariant aggregation functions are used — for example, mean, sum and maximum — in the message-passing and final prediction layers.

Expressivity

Imposing biases and symmetries controls the set of functions a model is not allowed to learn. By contrast, expressivity analysis looks at which set of functions a model is able to learn. Studying the expressiveness of GNNs provides an understanding of which patterns a model will and will not be able to capture, which is important for designing the best architecture and features to address the problem. In practice, an architecture should be chosen with an expressivity that captures the patterns critical for the task. An unjustified increase in the expressivity can lead to a worse generalization capacity due to overfitting.

Global expressivity. A necessary condition for a model to represent an arbitrary function on a graph is its ability to distinguish if two graphs are identical, a problem known as graph isomorphism. As no polynomial time algorithm is known to solve graph isomorphism for arbitrary graphs, there is currently no maximally expressive GNN whose runtime is polynomial in the number of nodes. As a result, researchers have studied the classes of graphs that GNN architectures can or cannot distinguish.

One key result for this analysis comes from the similarity[14,15] between message-passing layers in GNNs and the Weisfeiler–Leman isomorphism test, a classical algorithm in which each node is repeatedly assigned the hash of the representation of its neighbours. This analogy implies that standard GNNs are not able to distinguish graphs like the ones shown in Fig. 3b and, therefore, would predict them to have

Table 1 | Message-passing function of popular graph neural network architectures

Name	Message-passing	Ref.
GCN	$h^{(l+1)}=\sigma\left(\widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}h^{(l)}W+b\right)$	117
GAT	$h_i^{(l+1)}=\sigma\left(\sum_{j\in N_i}a(h_i^{(l)},h_j^{(l)})Wh_j^{(l)}\right)$	30
GIN	$h_i^{(l+1)}=\rho\left((1+\varepsilon)h_i^{(l)}+\sum_{j\in N(i)}h_j^{(l)}\right)$	14
MPNN	$h_i^{(l+1)}=\rho\left(h_i^{(l)},\sum_{j\in N_i}\mu(h_i^{(l)},h_j^{(l)})\right)$	73
PNA	$h_i^{(l+1)}=\rho\left(h_i^{(l)},\mathop{\Vert}_{\oplus\in\mathbb{A}}\mathop{\oplus}_{j\in N_i}\mu(h_i^{(l)},e_{ij},h_j^{(l)})\right)$	31

In the equations, $h^{(l)}$ Indicates the representations at layer l; A is the adjacency matrix, $A^*=A+IN$, $D^*ij=Pj\,A^*ij$; ρ and μ are learnable feedforward transformation functions; σ a simple nonlinearity; ε and W are a scalar and matrix parameter, respectively; a is a learned attention function; $\Vert$ represents concatenation; $\mathbb{A}$ is a set of aggregators. GAT, graph attention network; GCN, graph convolutional network; GIN, graph isomorphism network; MPNN, message passing neural network; PNA, principal neighbourhood aggregation.

the same properties. Several works have tried to extend the classes of distinguishable graphs by considering higher-order interactions[15] or features[16]. These expressivity improvements often come at the cost of greater computational complexity and longer runtimes.

Given that the shortcomings of message-passing arise from symmetries in the graphs, an alternative strategy to provably improve expressivity is to make the nodes more distinguishable, for example, by augmenting the input node features with random vectors[17,18] or positional encodings that indicate the global position of a node in the graph[19–23]. The most popular positional encoding features are the normalized eigenvectors of the Laplacian matrix, known in graph signal processing literature for their numerous desirable properties.

Global expressivity studies based on the Weisfeiler–Leman test have two drawbacks. First, they only consider the graph structure and ignore the node or edge features, which are an integral part of the input to the function defined with GNNs. Recent works have investigated the expressiveness properties of how GNNs treat feature information[24,25]. Second, Weisfeiler–Leman expressiveness does not provide a clear, interpretable understanding of a GNN's capacity. Increased expressiveness often does not correlate with improved empirical performance. For these reasons, several works have instead opted to study GNNs using lens of local expressivity.

Local expressivity. A related class of analysis looks at local patterns in the graphs that GNNs can and cannot detect. For example, GNNs are not able to count certain substructure types, such as cycles. However, rings are often critically important in molecules and cliques in social networks. Therefore, the initial node and edge features of GNNs are often augmented with number cycles to contain them[26]. Positional encodings can also help models recognize local motifs[21]. In practice, this creates a hybrid approach in which flexible inference of the deep learning model is augmented with hand-crafted features as inputs, because these are known to be relevant but cannot be captured by the architecture. Providing the features as part of the input enables the model to detect more complex patterns.

In knowledge graph completion, capturing logic inference patterns — such as composition pattern, hierarchy and mutual exclusion — is critical. Therefore, GNN architectures and embedding spaces are designed to model as many patterns as possible[27–29].

GNN variants

GNN architectures. Mathematically, the general message-passing layer can be written as:

$$h_i^{(l+1)}=\rho\left(h_i^{(l)},\bigoplus_{j\in N_i}\mu\left(h_i^{(l)},h_j^{(l)},e_{ij}\right)\right)$$

in which ρ and μ are, for example, learnable feedforward transformation functions and $\bigoplus$ is a predetermined aggregation function. Although hundreds of GNN architectures have been proposed, most can be described by the specific choice of aggregation and transformation functions. A summary of the functions used in common architectures is provided in Table 1.

Among these, graph attention network[30] and principal neighbourhood aggregation[31] propose two complementary strategies to improve the aggregation step and reduce bottlenecks. Graph attention network uses an attention mechanism to actively determine the weight given to each neighbour, enabling it to focus on the most relevant ones. By contrast, principal neighbourhood aggregation integrates multiple aggregation functions, meaning the nodes receive further information to increase the expressive power of the network.

Beyond simple message-passing. Many works have proposed ideas that go beyond this simple framework based on theoretical or empirical motivations. For example, a simple strategy shown to be particularly effective for molecular graphs is to update both the representation of nodes and edges, referred to as directional message passing neural network[10].

In certain domains, it is beneficial to decouple the computational graph that determines the information flow from the graph that underlies the data. Approaches to achieve this include rewiring the graph[32,33] to preserve a meaningful structure, while alleviating bottlenecks, or passing messages with multiple graphs[34].

Graph rewiring is also used to address the challenge of reasoning over long-range interactions: GNNs struggle to model patterns covering large distances over the graph; however, these can be helpful in several applications[35]. A popular approach to capturing long-range patterns is to process graphs with transformer-like architectures that operate on all pairs of nodes and use the original structure to bias the attention weights or provide a positional encoding to each node[36,37].

Flowing information via message-passing on the graph structure is not the only way to use the inductive bias from graph structures. New approaches to incorporate and parse graph structures have been proposed, borrowing ideas from topology and physics. Using topological concepts, graph structures are represented in terms of simplicial[38] or cell complexes[39], and message-passing interactions are generalized to these topological objects. Physics-inspired methods consider the graph structure as a discretization of a continuous manifold, in which the message-passing process is a diffusion partial differential equation on the manifold[40]. This framework enables graph structures to be treated as continuous objects, using ideas from the diffusion process in physical systems to improve message-passing over graphs[41–43].

Geometric graphs. In some domains, the nodes in the graph are embedded in the 3D space. Although these coordinates could be used as features of the nodes, they are typically treated separately, because they only provide a meaningful feature when analysed in relation to one another. Therefore, geometric graphs are modelled with specific types

of message-passing operators in which the messages are constructed and passed based on the relative position of the two nodes.

When working with graphs embedded in three dimensions, such as the 3D structures of a molecule, it is important to consider the symmetry of the task with respect to translations and rotations of the frame of reference (Fig. 3a). This translates into SE(3) invariance or equivariance, in which SE(3) is the special euclidean group in three dimensions, that is, the group of rotations and translations in three dimensions.

To design SE(3)-invariant architectures, the coordinates of the nodes cannot be taken as normal input features, because they would cause the model's output to change when the frame of reference is translated. Similarly, taking the relative vectors as edge features is problematic, as they change when the system is rotated. The easiest way to achieve rotation invariance is to extract only the relative distances between pairs of nodes and use these as edge features in message-passing[44,45].

However, only using distances in message-passing does not yield very expressive architectures. Similarly to arbitrary graphs, more powerful models are obtained using either higher-order representations or multi-hop interactions. Unlike general graphs, for which universality is unattainable due to the intrinsic challenge of graph isomorphism, the grounding of nodes in 3D space makes isomorphism easier on geometric graphs. Both strategies can yield architectures that are theoretically maximally expressive and are able to approximate any continuous equivariant function on a set of points in the 3D space[46].

In the higher-order strategy[11,47,48], the hidden representations of nodes contain normal SE(3)-invariant scalar features, SE(3)-equivariant vectors and higher-order representations. These more complex features can represent physical properties, such as forces and polarizability; however, they have to be handled with specific equivariant operations, such as tensor products. In the multi-hop interaction strategy[49,50], in addition to distances between pairs of nodes, the angles between pairs of connected edges and dihedral angles between three consecutive edges are used. These additional features enable complex relationships to be distinguished that cannot be easily captured when relying on simple distances.

Interpretability and uncertainty

Interpretability. Moving from a simple model based on hand-crafted features or rules to a deep learning solution comes at the cost of the degree of interpretability of the predictions. Instead of using human-interpretable rules, predictions are based on layers of transformations that produce representations without human-understandable meanings. This is also the case for GNNs; however, among architectures, they are inherently more interpretable and explainable, because they learn about relations between human-understandable entities from the nodes used to define the graph. For example, an inference based on a GNN's link prediction in a knowledge graph is easier to explain and interpret than the same inference made by an unstructured model. The additional structure of the graph-based problem formulation can be used for interpretability by inferring which node or subgraph of the input explains a prediction the most. To do so, researchers have developed several techniques similar to the general approaches for neural network interpretability but that also take into account the discreteness and symmetries of the graph structure.

Two of the most common strategies are gradient-based[51] or perturbation-based methods[52], both of which try to pinpoint the components of the input that most affects the output. An example of the latter strategy is GNNExplainer[53], which applies various modifications to the input data to determine which subgraph and features were the most important for the prediction. Another strategy is to build surrogate models[54], simpler, more interpretable architectures trained to reproduce the inputs and outputs of the base model. Finally, graph generation methods can build simple example structures to maximize the likelihood of a class under the model[55]. More detailed taxonomy and description of the different GNN interpretability approaches are provided in refs. 56,57.

Uncertainty estimation. Uncertainty estimation in machine learning determines how much a prediction can be trusted. Like interpretability, this is more difficult in a deep learning setting than in classical approaches. For GNNs, uncertainty quantification comes with unique challenges. For example, data uncertainty or epistemic uncertainty can arise from multiple sources with different impact magnitudes for node features or missing or incorrect edges. Similarly, how uncertainty propagates through layers and passed messages to produce a final prediction in GNNs is different from simpler architectures that are comparatively better studied for uncertainty quantification. These challenges mean that traditional deep learning uncertainty estimation methods fail when applied to GNNs in an inductive setting[58]. In the transductive setting, a major difficulty is the missing assumption on independent identically distributed samples. Without this, many of the general uncertainty estimation approaches do not apply. In practice, GNNs are underconfident[59] in the transductive setting. To address these issues, GNN have been developed with tailored techniques, such as custom Bayesian node updates, to disentangle epistemic and aleatoric uncertainty[60], and topology-dependent correction steps of the confidence[61,62].

Applications

With the abundance of graph-structured data in science and society, GNNs have found wide applicability, with meaningful impact in many fields. However, due to the range of tasks, it is crucial to consider application-specific information when selecting a model, as there is no one-size-fits-all GNN. An architecture should be chosen that best fits the application along multiple axes, such as scalability, expressivity and data efficiency. For instance, one axis is the trade-off between expressivity and memory usage, a core consideration for large protein–protein interaction graphs. On another axis, chemical priors — for instance, the importance of rings — are crucial to small-molecule property prediction[26]. Finally, in machine-learned interatomic potentials for molecular dynamics, inference speed is one of the main challenges[63].

Although standard GNNs can address many tasks adequately, there are cases in which simple solutions fail or cannot be used. These cases require additional insights to be built into the architecture. This section demonstrates this with literature examples highlighting some important GNN applications in the life and physical sciences.

Knowledge graphs

Knowledge graphs model relational data via nodes that represent different entities and directed edges that symbolize various relationships. For instance, in a biomedical knowledge graph, nodes might be diseases, drug molecules, proteins or viruses (Fig. 4a). The edges could encode relations about whether a drug cures a disease, a drug binds to a protein, a protein is relevant for a disease or similar.

To process knowledge graphs, specialized GNNs have been proposed[64,65] to handle the heterogeneous types of edges and nodes. Node embeddings from these architectures can predict the probability of unknown relations. In a biomedical context, for example, the

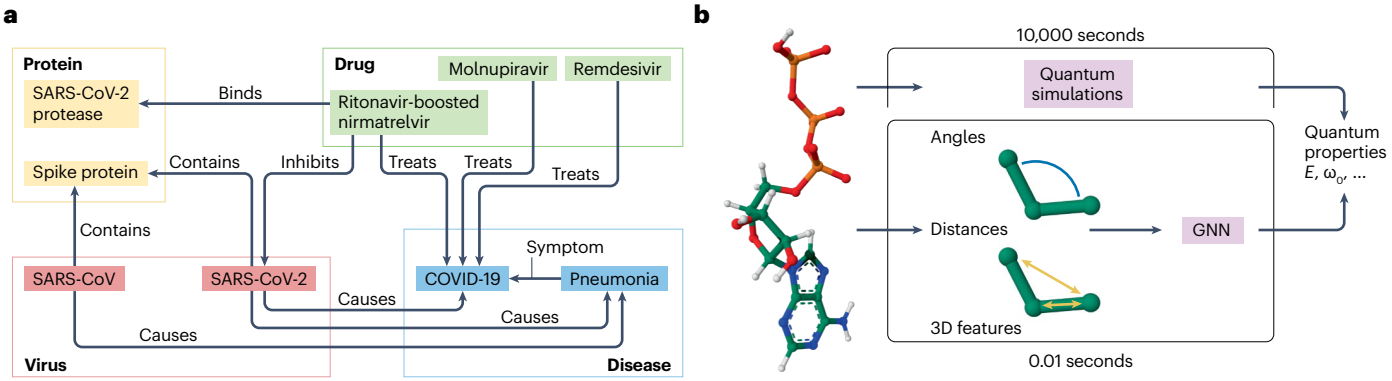

Fig. 4 | GNNs for knowledge graphs and molecular property prediction. **a**, An example of a biomedical knowledge graph with different types of interactions between entities (nodes) that are either proteins, drugs, viruses or diseases. **b**, Quantum property prediction with a graph neural network (GNN) as a representative task for molecular property prediction. Although accurate quantum simulations to estimate properties can take hours, GNNs have been successful at predicting quantum properties in fractions of seconds. *E*, potential energy; ω_0, vibrational mode frequency; SARS-CoV, severe acute respiratory syndrome coronavirus; SARS-CoV-2, severe acute respiratory syndrome coronavirus 2.

unknown relation could be whether an existing drug can be repurposed to treat additional diseases. In this drug discovery context, knowledge graphs offer the opportunity to integrate additional data from many modalities like drugs, phenotypes, diseases, disease exposure, genes or pathways, each with their own types of relations[7]. Outside the biomedical context, GNNs for knowledge graphs have heavily impacted recommender systems used in retail, advertisement and social media[66].

Molecular property prediction

An impactful application of GNNs is to predict (un)desirable properties of small molecules. A prominent example is ligand-based virtual screening, in which GNNs are trained to predict a property and scan large sets of molecules to identify candidates with the most favourable properties. For instance, a directional message passing neural network[10] combined with 200 additional molecule-level features was used[67] to predict a molecule's ability to inhibit *Escherichia coli* bacteria growth and helped discover a new antibiotic. In these settings, active learning is also used to refine the model's predictions based on different rounds of experimental validation.

Although the standard GNNs in Table 1 are often adequate for predicting properties like toxicity; absorption, distribution, metabolism, excretion (ADME); or synthesizability[68,69], their performance can be improved by including additional prior knowledge about molecules, such as the importance of rings. Therefore, cycles and other subgraph counts are often added to initial node and edge features. Laplacian-based positional encodings have been successful with a standard GNN architecture for predicting mass spectra[70]. Other architecture improvements that have shown promise in property prediction are directional graph networks[20], in which positional encodings improve message guidance, and subgraph aggregation[71], in which messages are passed over subsets of the molecular graph. Using information about a molecule's synthesis or generation path as an input can provide additional signals for better generalization and data efficiency[72].

Architecture specializations were necessary to improve GNNs for predicting quantum mechanical properties (visualized in Fig. 4b), one of the first message-passing applications[73]. Graph transformers were used to pass messages between all nodes while retaining the graph structure. The graph structure was included by encoding it as an initial node feature or using simultaneous message-passing layers for the molecular graph[34]. Large transformer architectures are particularly well-suited to utilize the increasing amounts of data generated with quantum simulations, from GEOM-QM9 and GEOM-DRUGS[74] to PCQM4Mv2 (ref. 12). For quantum properties, GNNs are also able to obtain electronic structures via variational quantum Monte Carlo, increasing speeds and bringing a new level of generalizability to the field[75,76].

Graph generation

GNNs are fundamental building blocks in generative models on graphs. Graph generation has several compelling applications. For example, to find an effective drug for a particular disease, billions of small molecules could be virtually screened, but this would still only access a small fraction of the synthesizable molecular space. Direct generation of candidate molecules with certain desired properties could drastically reduce computational costs, while expanding the number of accessible compounds.

However, although the generation of images (fixed-size vectors) and text (ordered sequence of tokens) is successful with deep learning approaches, the variability — different numbers of nodes and edges — and symmetries of graphs render the generation process particularly challenging. Initial approaches were largely based on the generative modelling frameworks of variational auto encoders[77] or generative adversarial networks[78], both of which involve learning the transformation of a fixed-size random vector into a graph. Different approaches to achieve this complex mapping include building a single, large, fixed-size adjacency matrix and masking it[79]; iteratively building a graph by adding nodes or subgraphs[80,81] (Fig. 5a); or starting from a fixed reference graph and learning to modify it[82]. Diffusion-based models, which learn to gradually map randomly sampled graphs to those of interest, can provide large improvements across different domains[83,84]. A particular challenge for graph generation is performance evaluation.

Biophysical structure, dynamics and interactions

3D GNNs can model biophysical structures as they are able to represent 3D point clouds, for example protein residues, and have a physically realistic prior that local interactions are the most relevant, whereas distant forces decay rapidly.

In rational protein design, message-passing-based tools are critical to tackling inverse folding[85], in which the aim is to reconstruct the amino acid sequence from a 3D point cloud representing a backbone structure. Similar architectures have also been applied to predict the strength of the interaction between molecules. For instance, PiGNet[86] predicts the affinity between a molecule and the protein it is bound to. For multiple additional drug design-related approaches, GNNs have been used as the base architecture for generative models over molecular structures. Notable examples include generating the most likely 3D structures of small molecules[87,88] (conformer generation; Fig. 5b); the distribution of protein structures[89,90] (protein folding); structures used by small molecules to bind to proteins[91] (molecular docking; Fig. 5b); or structures of novel proteins[92,93] (rational protein design).

Another common approach to determining the flexibility of biophysical structures is to learn their dynamics and increase their simulation speed. In this setting, GNNs are used as molecular potentials that are trained to predict the energy[44,49,50,63] of a given atomic structure. Afterwards, the gradient, the predicted force, is used in the simulation to update the atom positions. Other methods directly predict future atom positions[94] or speed up molecular dynamics simulations by generating abstracted, lower dimensional, coarse-grained molecular representations[95]. GNNs can also undo coarse-grainings in a generative fashion[96].

Reproducibility and data deposition

Data releases and good reproducibility practices have helped develop GNNs. These have been partially driven by standardized benchmarks, such as the OGB[12] and Therapeutic Data Commons[97], which require code to reproduce results to be published. Despite this progress, lack of data is an issue for many life science applications, because data acquisition is more expensive and diverse than in computer vision or natural language processing, in which scraping the internet often suffices for data collection. These challenges highlight the value of collating and open-sourcing more data, alongside developing methods for the low data regime.

Data sources and benchmarks

Benchmark suites — such as OGB, Therapeutic Data Commons or the Open Catalyst Project — provide collections of datasets with standardized train/validation/test sets and evaluation metrics. They come with PyTorch Geometric and Deep Graph Library interfaces for data loaders and evaluation metrics to set up experiments in a comparable and reproducible manner, with online leaderboards to compare state-of-the-art methods. In drug discovery, the Therapeutic Data Commons data collection is notable, with a wide range of tasks, from protein–ligand affinity to retrosynthesis and toxicity prediction.

Another large-scale data collection effort is the Protein Data Bank, which contains over 200,000 protein 3D atomic structures and has enabled many developments in machine learning for structural biology. Multiple protein structures occur as complexes with small molecules, and PDBBind is an effort to extract and curate structures from the Protein Data Bank with publicly available binding affinity values. A large source of bioactivity data is ChEMBL, which has activity measurements for 2.4 million compounds. Drawing from these sources is the precision medicine knowledge graph[7], which has relationships between 129,000 nodes, with types ranging from diseases, drugs and genes to anatomical regions and disease exposures. Finally, there are multiple sources of protein–protein interaction graphs, and more information can be found in ref. 98, which surveys and compares 16 databases.

Limitations and optimizations

Evaluation

The variety of tasks that can be addressed with GNNs means there is ambiguity in evaluation criteria and a danger of using irrelevant metrics. This is particularly relevant for generative tasks, for which the goodness of an output is difficult to quantify. When generating new drug-like molecules, simple metrics may include chemical validity, synthesizability, diversity and distance from the training data. However, to evaluate more complex biological phenomena, such as biological activity or toxicity, computational estimators can be inaccurate and misleading.

Data dependence

Although GNNs are state of the art for many tasks on graph-structured data, they are not the universal best option due to several technical and data limitations. For instance, for some molecular property prediction tasks, molecular fingerprints offer better performance[10,99],

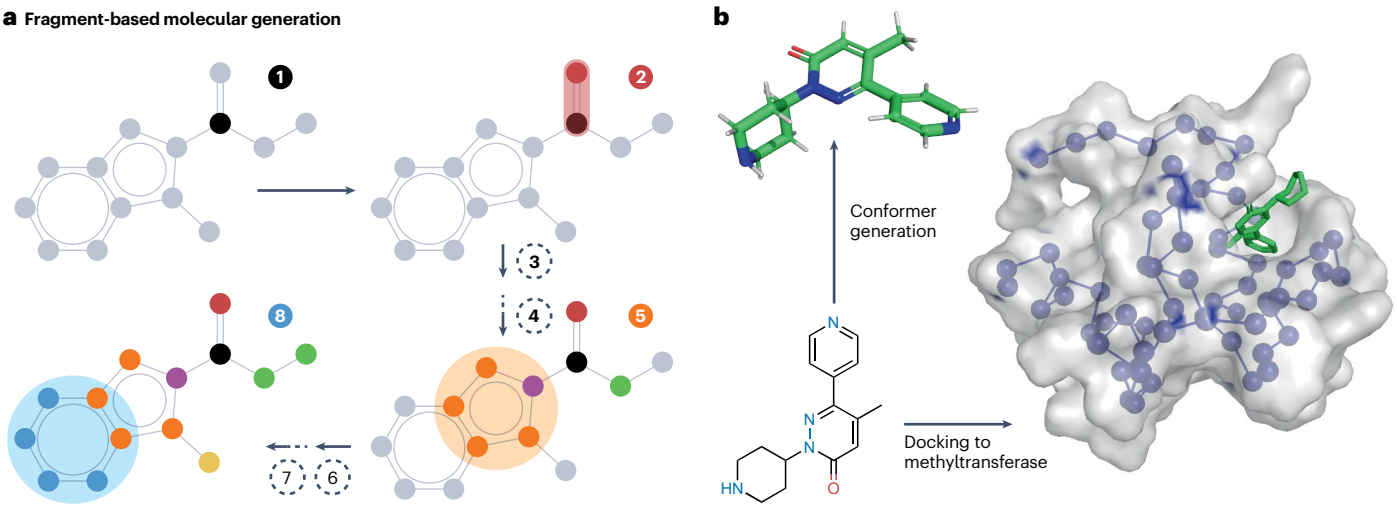

Fig. 5 | Examples of GNNs for generative modelling. a, Example of fragment-based molecular generation process similar to ref. 80. **b**, Representation of the conformer generation and docking tasks. For docking tasks, the target protein to which the ligand is docked is visualized with both the amino acid sequence, which is how many graph neural network (GNN)-based methods represent it, and the surface. Protein structure from Protein Data Bank 6G29.

Glossary

Big O notation
Notation used in complexity theory to indicate how the worst-case runtime of an algorithm increases as the size of the input increases.

Composition pattern
A simple example composition pattern is if molecule A binds to protein B and protein B is involved in the mechanism of disease C, then A is a potential candidate for C.

Deep learning
Subset of machine learning that uses artificial neural network models with multiple layers learning to automatically extract features and complex patterns from data.

Embeddings
Arrays of numbers produced by a deep learning model abstractly capture a model's understanding of an object.

Features
Information about the object under analysis that is passed as inputs to the model.

Knowledge graph completion
Task in which missing information in a knowledge graph is predicted based on existing relationships and patterns within the graph.

Message-passing layer
Fundamental component of graph neural networks that iteratively aggregates and updates the features from neighbouring nodes, enabling the propagation of information throughout the graph structure.

Planar graphs
A planar graph is one that can be drawn on a 2D page without edges crossing each other.

ReLU
The rectified linear unit (ReLU) is the most common type of non-linear function used in neural networks and has the simple form $\mathrm{ReLU}(x)=\max(0,x)$.

Representations
Arrays of numbers that capture attributes of an object.

ROC-AUC
(Area under the curve of the receiver operator characteristic). A measure of the precision of a binary classifier that is informative in settings with unbalanced classes.

Scaffold
Core substructures within molecular graphs shared by multiple compounds that often have similar properties.

Transductive task
Setting that involves making predictions at inference time on a partially labelled graph, for a subset of the nodes within the graph. Models trained in a transductive setting do not generalize to other graphs.

Uncertainty
Uncertainty refers to the lack of confidence or precision in a model's prediction. Taking this ambiguity into account is often important in real-world applications of machine learning models.

scalability and interpretability. The generalization capacity of fingerprints is particularly helpful in the low data regime, in which large GNNs can overfit easily. For example, if the training data contain only several hundred examples with very few positives, a GNN with many parameters will likely overfit, as the model does not have enough supervision to learn which features are robust and which are spurious correlations. The lack of data in many areas of the life sciences is a big limitation to the application of GNNs. Building architectures that incorporate prior knowledge about the problem can alleviate this challenge, as the model does not have to learn these aspects from the scarce data. Another core method for addressing a lack of data is pretraining.

Pretraining. In pretraining approaches, a model is trained on a related task, for which more data are available, to teach the model to extract relevant features. Subsequently, the limited labelled data are used to learn how to recombine those features for the task at hand, referred to as fine-tuning. Pretraining GNNs has shown some success. In quantum chemistry-related tasks[100,101], large datasets of molecular physical properties obtained via expensive calculations are used to pretrain expressive models. However, graph pretraining remains challenging for many domains, partly due to the failure of self-supervised learning approaches.

Self-supervised learning. Pretraining of large models in an unsupervised fashion has driven incredible progress for text[102,103], protein sequences[104] and images[105–107]. These methods use techniques like contrastive learning or masking to design synthetic tasks aimed at building expressive representations from large amounts of unlabelled data. However, direct application to molecules and other graph-structured data has not been successful[108], likely due to limited relevance.

For example, predicting a masked word in a sentence requires meaningful digestion of its context, which is helpful for understanding natural language. In a molecule, however, it is almost trivial to infer an atom's element given its context due to the number of possible bonds. Similarly, in computer vision, contrastive learning helps models become invariant to data augmentations or distortions. For molecules, however, deleting an atom results in completely different chemical properties.

Technical limitations

Oversmoothing. Oversmoothing is a widely known limitation of GNNs, in which individual node features become nearly identical as the number of GNN layers increases, because each message-passing layer behaves as a graph-smoothing operator[109]. This phenomenon prevents very deep networks, which are prevalent in other domains. Several methods to alleviate oversmoothing have been proposed. For instance, JKNets[110] introduce skip connections in which the initial node features are added to the node features after every layer, Graff[111] biases the information flow to alleviate oversmoothing and Gradient Gating[112] uses a gating mechanism to control local information flow in the graph. Although these approaches afford improvements, there is still no clear solution to this challenge.

Oversquashing. Oversquashing refers to the topology of the graph causing bottlenecks in information flow, leading to little signal and influence between distant nodes[113]. At the bottlenecks, many nodes' features must be compressed into a single node's representation, which limits the GNNs' ability to capture long-range dependencies. Oversquashing has seen multiple analyses, with proposed solutions, such as adding a virtual global node or rewiring the graph[114], but it remains an open problem.

Outlook

GNNs are limited in their expressiveness, interpretability and data efficiency, with practitioners often partially relying on feature engineering. However, to overcome these challenges, there is active research, some of which is presented in this section.

Although GNNs are fundamentally bounded in expressivity by the intrinsic complexity of distinguishing general graphs, in practical domains arbitrary graphs are rarely encountered. Molecules and road networks, for example, are almost exclusively planar graphs in which

it is possible to build fully expressive architectures[115]. Considerations about particular domains will need to be integrated into efficient architectures that do not suffer from bottlenecks.

Methods to interpret GNNs are currently limited to identifying nodes or substructures that most influence a decision. Usually, this is not enough to truly understand the model's reasoning or build surrogate, less-expressive models. Instead, using domain knowledge and multi-modal integrations, interpretability can be directly built into the task the model optimizes for. For example, to predict if a molecule is toxic, instead of framing the task as a simple binary classification, the model could be trained to predict which human proteins the ligand binds to and whether that interaction causes adverse side effects. This prediction is substantially more interpretable and experimentally verifiable than a binary toxicity classification.

Finally, an underexplored GNN application in the life sciences is modelling dynamic graphs. Many biological phenomena with a graph structure change over time. For instance, brain activity profiles can be modelled as brain networks with signals for nodes that evolve over time, or disease spread can be modelled as a dynamic graph in which better forecasts can have large positive impacts. Temporal graph networks are well researched for applications outside of the life sciences[116]. A promising direction could be applying them to life science problems.

Despite these limitations, GNNs have the capacity to strongly impact many applications in the life sciences and beyond. With new state-of-the-art approaches in fields from drug and antibiotic discovery and traffic prediction to structural biology and recommendation systems, it is expected that the application of GNNs, in their current and future forms, will enable discoveries and the development of a wide variety of new products.

Code availability

Example code can be found at https://github.com/HannesStark/GNN-primer/blob/main/GNN-primer_HIV_classification.ipynb.

Published online: 07 March 2024

References

1. Gori, M., Monfardini, G. & Scarselli, F. A new model for learning in graph domains. In *Proceedings 2005 IEEE International Joint Conference Neural Networks* 729–734 (IEEE, 2005).
2. Merkwirth, C. & Lengauer, T. Automatic generation of complementary descriptors with molecular graph networks. *J. Chem. Inf. Model.* **45**, 1159–1168 (2005).
3. Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M. & Monfardini, G. The graph neural network model. *IEEE Trans. Neural Netw.* **20**, 61–80 (2008).
   Although the genealogy of the development is multifaced, this is often considered as the first instance of GNNs.
4. Bronstein, M. M., Bruna, J., Cohen, T. & Veličković, P. Geometric deep learning: grids, groups, graphs, geodesics, and gauges. Preprint at https://doi.org/10.48550/arXiv.2104.13478 (2021).
   Book with a very comprehensive introduction to the theoretical aspects behind GNNs and other geometric deep learning architectures.
5. Jegelka, S. Theory of graph neural networks: representation and learning. Preprint at https://doi.org/10.48550/arXiv.2204.07697 (2022).
6. Morgan, H. L. The generation of a unique machine description for chemical structures-a technique developed at chemical abstracts service. *J. Chem. Doc.* **5**, 107–113 (1965).
7. Chandak, P., Huang, K. & Zitnik, M. Building a knowledge graph to enable precision medicine. *Sci. Data* **10**, 67 (2023).
8. Fey, M. & Lenssen, J. E. Fast graph representation learning with PyTorch Geometric. Preprint at https://doi.org/10.48550/arXiv.1903.02428 (2019).
   PyTorch Geometric is the most widely used library to develop GNNs.
9. Wang, M. et al. Deep Graph Library: a graph-centric, highly-performant package for graph neural networks. Preprint at https://doi.org/10.48550/arXiv.1909.01315 (2019).
10. Yang, K. et al. Analyzing learned molecular representations for property prediction. *J. Chem. Inf. Model.* **59**, 3370–3388 (2019).
11. Geiger, M. & Smidt, T. e3nn: Euclidean neural networks. Preprint at https://doi.org/10.48550/arXiv.2207.09453 (2022).
12. Hu, W. et al. Open Graph Benchmark: datasets for machine learning on graphs. *Adv. Neural Inf. Process. Syst.* 22118–22133 (NeurIPS Proceedings, 2020).
    OGB is the most widely used benchmark for GNNs with a wide variety of datasets, each with its own leaderboard.
13. Dummit, D. S. & Foote, R. M. *Abstract algebra* 7th edn (Wiley, 2004).
14. Xu, K., Hu, W., Leskovec, J. & Jegelka, S. How powerful are Graph Neural Networks? In *International Conference on Learning Representations* (ICLR, 2019).
    To our knowledge, this work, concurrently with [Mor+19], was the first to propose and use the analogy of GNNs to WL isomorphism test to study their expressivity.
15. Morris, C. et al. Weisfeiler and Leman go neural: higher-order graph neural networks. *Proc. AAAI Conf. Artif. Intell.* **33**, 4602–4609 (2019).
16. Vignac, C., Loukas, A. & Frossard, P. Building powerful and equivariant graph neural networks with structural message-passing. *Adv. Neural Inf. Process. Syst.* **33**, 14143–14155 (2020).
17. Abboud, R., Ceylan, I.I., Grohe, M. & Lukasiewicz, T. The surprising power of graph neural networks with random node initialization. In *30th International Joint Conferences on Artificial Intelligence* 2112–2118 (International Joint Conferences on Artificial Intelligence Organization, 2021).
18. Sato, R., Yamada, M. & Kashima, H. Random features strengthen graph neural networks. In *Proceedings of the 2021 SIAM International Conference on Data Mining* 333–341 (Society for Industrial and Applied Mathematics, 2021).
19. Dwivedi, V. P. et al. Benchmarking graph neural networks. *J. Mach. Learn. Res.* **24**, 1–48 (2023).
20. Beaini, D. et al. Directional graph networks. In *Proceedings of the 38th International Conference on Machine Learning* 748–758 (PMLR, 2021).
21. Lim, D. et al. Sign and basis invariant networks for spectral graph representation learning. In *International Conference on Learning Representations* (ICLR, 2023).
22. Keriven, N. & Vaiter, S. What functions can Graph Neural Networks compute on random graphs? The role of Positional Encoding. Preprint at https://doi.org/10.48550/arXiv.2305.14814 (2023).
23. Zhang, B., Luo, S., Wang, L. & He, D. Rethinking the expressive power of GNNs via graph biconnectivity. In *International Conference on Learning Representations* (ICLR, 2023).
24. Di Giovanni, F. et al. How does over-squashing affect the power of GNNs? Preprint at https://doi.org/10.48550/arXiv.2306.03589 (2023).
25. Razin, N., Verbin, T. & Cohen, N. On the ability of graph neural networks to model interactions between vertices. In *37th Conference on Neural Information Processing Systems* (NeurIPS, 2023).
26. Bouritsas, G., Frasca, F., Zafeiriou, S. & Bronstein, M. M. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**, 657–668 (2023).
27. Sun, Z., Deng, Z.-H., Nie, J.-Y. & Tang, J. RotatE: knowledge graph embedding by relational rotation in complex space. Preprint at https://doi.org/10.48550/arXiv.1902.10197 (2019).
28. Abboud, R., Ceylan, I., Lukasiewicz, T. & Salvatori, T. BoxE: a box embedding model for knowledge base completion. *Adv. Neural Inf. Process. Syst.* **33**, 9649–9661 (2020).
29. Pavlović, A. & Sallinger, E. ExpressivE: a spatio-functional embedding for knowledge graph completion. In *International Conference on Learning Representations* (ICLR, 2023).
30. Veličković, P. et al. Graph attention networks. In *International Conference on Learning Representations* (ICLR, 2017).
    Graph attention networks are the first application of the idea of attention to graphs, and they are one of the most widely used architectures to date.
31. Corso, G., Cavalleri, L., Beaini, D., Liò, P. & Veličković, P. Principal neighbourhood aggregation for graph nets. *Adv. Neural Inf. Process. Syst.* **33**, 13260–13271 (2020).
32. Gasteiger, J., Weißenberger, S. & Günnemann, S. Diffusion improves graph learning. *Adv. Neural Inf. Process. Syst.* **32**, 13366–13378 (2019).
33. Gutteridge, B., Dong, X., Bronstein, M. & Di Giovanni, F. DRew: dynamically rewired message passing with delay. In *International Conference on Machine Learning* (eds Krause, A. et. al.) 12252–12267 (ICML, 2023).
34. Rampášek, L. et al. Recipe for a general, powerful, scalable graph transformer. *Adv. Neural Inf. Process. Syst.* **35**, 14501–14515 (2022).
35. Dwivedi, V. P. et al. Long range graph benchmark. *Adv. Neural Inf. Process. Syst.* **35**, 22326–22340 (2022).
36. Dwivedi, V. P. & Bresson, X. A generalization of transformer networks to graphs. Preprint at https://doi.org/10.48550/arXiv.2012.09699 (2020).
37. Kreuzer, D., Beaini, D., Hamilton, W., Létourneau, V. & Tossou, P. Rethinking graph transformers with spectral attention. *Adv. Neural Inf. Process. Syst.* **34**, 21618–21629 (2021).
38. Bodnar, C. et al. Weisfeiler and Lehman go topological: message passing simplicial networks. In *Proceedings of the 38th International Conference on Machine Learning* (eds Meila, M. & Zhang, T.) 1026–1037 (PMLR, 2021).
39. Bodnar, C. et al. Weisfeiler and Lehman go cellular: cw networks. *Adv. Neural Inf. Process. Syst.* **34**, 2625–2640 (2021).
40. Chamberlain, B. et al. Grand: graph neural diffusion. In *Proceedings of the 38th International Conference on Machine Learning* (eds Meila, M. & Zhang, T.) 1407–1418 (PMLR, 2021).
41. Chamberlain, B. et al. Beltrami flow and neural diffusion on graphs. *Adv. Neural Inf. Process. Syst.* **34**, 1594–1609 (2021).
42. Di Giovanni, F., Rowbottom, J., Chamberlain, B. P., Markovich, T. & Bronstein, M. M. Graph neural networks as gradient flows. Preprint at https://doi.org/10.48550/arXiv.2206.10991 (2022).

43. Rusch, T. K., Chamberlain, B., Rowbottom, J., Mishra, S. & Bronstein, M. Graph-coupled oscillator networks. In *Proceedings of the 39th International Conference on Machine Learning* (eds Chaudhuri, K. et al.) 18888–18909 (PMLR, 2022).
44. Schütt, K. et al. SchNet: a continuous-filter convolutional neural network for modeling quantum interactions. In *NIPS'17: Proceedings of the 31st International Conference on Neural Information Processing Systems* (eds von Luxburg, U. et al.) 992–1002 (Curran Associates Inc., 2017).
SchNet is one of the earliest and most prominent examples of SE(3)-invariant GNNs.
45. Satorras, V. G., Hoogeboom, E. & Welling, M. E(n) equivariant graph neural networks. In *Proceedings of the 38th International Conference on Machine Learning* (eds Meila, M. & Zhang, T.) 9323–9332 (PMLR, 2021).
46. Dym, N. & Maron, H. On the universality of rotation equivariant point cloud networks. In *International Conference on Learning Representations* (ICLR, 2021).
47. Thomas, N. et al. Tensor field networks: rotation- and translation-equivariant neural networks for 3D point clouds. Preprint at https://doi.org/10.48550/arXiv.1802.08219 (2018).
48. Jing, B., Eismann, S., Suriana, P., Townshend, R. J. & Dror, R. Learning from protein structure with geometric vector perceptrons. In *International Conference on Learning Representations* (ICLR, 2021).
49. Gasteiger, J., Groß, J. & Günnemann, S. Directional message passing for molecular graphs. In *Adv. Neural Inf. Process. Syst.* (NeurIPS, 2020).
50. Gasteiger, J., Becker, F. & Günnemann, S. GemNet: universal directional graph neural networks for molecules. *Adv. Neural Inf. Process. Syst.* **34**, 6790–6802 (2021).
51. Baldassarre, F. & Azizpour, H. Explainability techniques for graph convolutional networks. Preprint at https://doi.org/10.48550/arXiv.1905.13686 (2019).
52. Schlichtkrull, M. S., De Cao, N. & Titov, I. Interpreting graph neural networks for NLP with differentiable edge masking. In *International Conference on Learning Representations* (ICLR, 2021).
53. Ying, Z., Bourgeois, D., You, J., Zitnik, M. & Leskovec, J. GNNExplainer: generating explanations for graph neural networks. *Adv. Neural Inf. Process. Syst.* **32**, 9240–9251 (2019).
54. Huang, Q., Yamada, M., Tian, Y., Singh, D. & Chang, Y. GraphLIME: local interpretable model explanations for graph neural networks. *IEEE Trans. Knowl. Data Eng.* **35**, 6968–6962 (2023).
55. Yuan, H., Tang, J., Hu, X. & Ji, S. XGNN: towards model-level explanations of graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* 430–438 (2020).
56. Yuan, H., Yu, H., Gui, S. & Ji, S. Explainability in graph neural networks: a taxonomic survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**, 5782–5799 (2022).
57. Kakkad, J., Jannu, J., Sharma, K., Aggarwal, C. & Medya, S. A survey on explainability of graph neural networks. Preprint at https://doi.org/10.48550/arXiv.2306.01958 (2023).
58. Hirschfeld, L., Swanson, K., Yang, K., Barzilay, R. & Coley, C. W. Uncertainty quantification using neural networks for molecular property prediction. *J. Chem. Inf. Model.* **60**, 3770–3780 (2020).
59. Hsu, H. H.-H., Shen, Y., Tomani, C. & Cremers, D. What makes graph neural networks miscalibrated? In *Adv. Neural Inf. Process. Syst.* (NeurIPS, 2022).
60. Stadler, M., Charpentier, B., Geisler, S., Zügner, D. & Günnemann, S. Graph posterior network: Bayesian predictive uncertainty for node classification. *Adv. Neural Inf. Process. Syst.* **34**, 18033–18048 (2021).
61. Wang, X., Liu, H., Shi, C. & Yang, C. Be confident! towards trustworthy graph neural networks via confidence calibration. *Adv. Neural Inf. Process. Syst.* **34**, 23768–23779 (2021).
62. Huang, K., Jin, Y., Candes, E. & Leskovec, J. Uncertainty quantification over graph with conformalized graph neural networks. Preprint at https://doi.org/10.48550/arXiv.2305.14535 (2023).
63. Batzner, S. et al. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nat. Commun.* **13**, 2453 (2022).
64. Schlichtkrull, M. S. et al. Modeling relational data with graph convolutional networks. In *The Semantic Web. ESWC 2018. Lecture Notes in Computer Science* (eds Gangemi, A. et al.) 593–607 (Springer, Cham, 2018).
65. Sun, Q. et al. SUGAR: subgraph neural network with reinforcement pooling and self-supervised mutual information mechanism. In *WWW '21: Proceedings of the Web Conference 2021* (eds Leskovec, J. et al.) 2081–2091 (Association for Computing Machinery, 2021).
66. Sharma, K. et al. A survey of graph neural networks for social recommender systems. Preprint at https://doi.org/10.48550/arXiv.2212.04481 (2022).
67. Stokes, J. M. et al. A deep learning approach to antibiotic discovery. *Cell* **180**, 688–702.e13 (2020).
Discovery of a novel antibiotic, halicin, via GNNs, one of the most prominent examples of the application of GNNs to scientific discovery.
68. Feinberg, E. N., Joshi, E., Pande, V. S. & Cheng, A. C. Improvement in ADMET prediction with multitask deep featurization. *J. Med. Chem.* **63**, 8835–8848 (2020).
69. Peng, Y. et al. Enhanced graph isomorphism network for molecular ADMET properties prediction. *IEEE Access* **8**, 168344–168360 (2020).
70. Murphy, M. et al. Efficiently predicting high resolution mass spectra with graph neural networks. In *Proceedings of the 40th International Conference on Machine Learning* (eds Krause, A. et al.) 25549–25562 (PMLR, 2023).
71. Bevilacqua, B. et al. Equivariant subgraph aggregation networks. In *International Conference on Learning Representations* (ICLR, 2022).
72. Guo, M. et al. Hierarchical grammar-induced geometry for data-efficient molecular property prediction. In *Proceedings of the 40th International Conference on Machine Learning* (eds Krause, A. et al.) 12055–12076 (PMLR, 2023).
73. Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O. & Dahl, G. E. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning* (eds Precup, D. & Teh, Y. W.) 1263–1272 (PMLR, 2017).
To our knowledge, this paper is the first to formalize the idea of message passing as presented in this Primer and proposes applications of GNNs to quantum chemistry, which remains one of the scientific fields in which GNNs have seen most applications.
74. Axelrod, S. & Gómez-Bombarelli, R. GEOM, energy-annotated molecular conformations for property prediction and molecular generation. *Sci. Data* **9**, 185 (2022).
75. Hermann, J., Schätzle, Z. & Noé, F. Deep-neural-network solution of the electronic Schrödinger equation. *Nat. Chem.* **12**, 891–897 (2020).
76. Gao, N. & Günnemann, S. Generalizing neural wave functions. In *International Conference on Machine Learning* 10708–10726 (ICML, 2023).
77. Kingma, D. P. & Welling, M. Auto-encoding variational bayes. In *International Conferece on Learning Representations* (ICLR, 2014).
78. Goodfellow, I. et al. Generative adversarial networks. *Commun. ACM* **63**, 139–144 (2020).
79. Mitton, J., Senn, H. M., Wynne, K. & Murray-Smith, R. A graph VAE and graph transformer approach to generating molecular graphs. Preprint at https://doi.org/10.48550/arXiv.2104.04345 (2021).
80. Jin, W., Barzilay, R. & Jaakkola, T. Junction tree variational autoencoder for molecular graph generation. In *Proceedings of the 35th International Conference on Machine Learning* (eds Dy, J. & Krause, A.) 2323–2332 (PMLR, 2018).
81. Jin, W., Barzilay, R. & Jaakkola, T. Hierarchical generation of molecular graphs using structural motifs. In *Proceedings of the 37th International Conference on Machine Learning* (eds Daumé, H. & Singh, A.) 4839–4848 (PMLR, 2020).
82. Vignac, C. & Frossard, P. Top-N: equivariant set and graph generation without exchangeability. In *International Conference on Learning Representations* (ICLR, 2022).
83. Jo, J., Lee, S. & Hwang, S. J. Score-based generative modeling of graphs via the system of stochastic differential equations. In *Proceedings of the 39th International Conference on Machine Learning* (eds Chaudhuri, K. et al.) 10362–10383 (PMLR, 2022).
84. Vignac, C. et al. DiGress: discrete denoising diffusion for graph generation. In *International Conference on Learning Representations* (ICLR, 2023).
85. Dauparas, J. et al. Robust deep learning–based protein sequence design using ProteinMPNN. *Science* **378**, 49–56 (2022).
86. Moon, S., Zhung, W., Yang, S., Lim, J. & Kim, W. Y. PIGNet: a physicsinformed deep learning model toward generalized drug–target interaction predictions. *Chem. Sci.* **13**, 3661–3673 (2022).
87. Xu, M. et al. GeoDiff: a geometric diffusion model for molecular conformation generation. In *International Conference on Learning Representations* (ICLR, 2022).
88. Jing, B., Corso, G., Chang, J., Barzilay, R. & Jaakkola, T. S. Torsional diffusion for molecular conformer generation. In *Adv. Neural Inf. Process. Syst.* (eds Sanmi, K. et al.) (NeurIPS, 2022).
89. Ingraham, J., Riesselman, A., Sander, C. & Marks, D. Learning protein structure with a differentiable simulator. In *International Conference on Learning Representations* (ICLR, 2019).
90. Jing, B. et al. EigenFold: generative protein structure prediction with diffusion models. Preprint at https://doi.org/10.48550/arXiv.2304.02198 (2023).
91. Corso, G., Stärk, H., Jing, B., Barzilay, R. & Jaakkola, T. S. DiffDock: diffusion steps, twists, and turns for molecular docking. In *International Conference on Learning Representations* (ICLR, 2023).
92. Ingraham, J. et al. Illuminating protein space with a programmable generative model. *Nature* **623**, 1070–1078 (2023).
93. Watson, J. L. et al. De novo design of protein structure and function with RFdiffusion. *Nature* **620**, 1089–1100 (2023).
94. Fu, X., Xie, T., Rebello, N. J., Olsen, B. D. & Jaakkola, T. Simulate time-integrated coarse-grained molecular dynamics with geometric machine learning. Preprint at https://doi.org/10.48550/arXiv.2204.10348 (2022).
95. Wang, W. et al. Generative coarse-graining of molecular conformations. In *International Conference on Machine Learning* 23213–23236 (ICML, 2022).
96. Yang, S. & Gomez-Bombarelli, R. Chemically transferable generative backmapping of coarse-grained proteins. In *Proceedings of the 40th International Conference on Machine Learning* (eds Krause, A. et al.) 39277–39298 (PMLR, 2023).
97. Huang, K. et al. Therapeutics data commons: machine learning datasets and tasks for drug discovery and development. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021* (NeurIPS, 2021).
98. Bajpai, A. K. et al. Systematic comparison of the protein-protein interaction databases from a user's perspective. *J. Biomed. Inform.* **103**, 103380 (2020).
99. Tripp, A., Bacallado, S., Singh, S. & Hernández-Lobato, J. M. Tanimoto random features for scalable molecular machine learning. In *Adv. Neural Inf. Process. Syst.* (NeurIPS, 2023).
100. Stärk, H. et al. 3D Infomax improves GNNs for molecular property prediction. In *Proceedings of the 39th International Conference on Machine Learning* (eds Chaudhuri, K. et al.) 20479–20502 (PMLR, 2022).
101. Thakoor, S. et al. Large-scale representation learning on graphs via bootstrapping. In *International Conference on Learning Representations* (ICLR, 2022).

102. Devlin, J., Chang, M., Lee, K. & Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (eds Burstein, J. et al.) 4171–4186 (Association for Computational Linguistics, 2019).
103. Brown, T. et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **33**, 1877–1901 (2020).
104. Lin, Z. et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* **379**, 1123–1130 (2023).
105. Dosovitskiy, A. et al. An image is worth 16x16 words: transformers for image recognition at scale. In *International Conference on Learning Representations* (ICLR, 2021).
106. Misra, I. & van der Maaten, L. Self-supervised learning of pretext-invariant representations. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 6707–6717 (IEEE, 2020).
107. He, K., Fan, H., Wu, Y., Xie, S. & Girshick, R. Momentum Contrast for unsupervised visual representation learning. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 9726–9735 (IEEE, 2020).
108. Liu, Y. et al. Graph self-supervised learning: a survey. *IEEE Trans. Knowl. Data Eng.* **35**, 5879–5900 (2023).
109. Rusch, T. K., Bronstein, M. M. & Mishra, S. A survey on oversmoothing in graph neural networks. Preprint at https://doi.org/10.48550/arXiv.2303.10993 (2023).
110. Xu, K. et al. Representation learning on graphs with jumping knowledge networks. In *Proceedings of the 35th International Conference on Machine Learning* (eds Dy, J. & Krause, A.) 5453–5462 (PMLR, 2018).
111. Di Giovanni, F., Rowbottom, J., Chamberlain, B. P., Markovich, T. & Bronstein, M. M. Understanding convolution on graphs via energies. In *Transact. Mach. Learn. Res.* 2835–8856 (2023).
112. Rusch, T. K., Chamberlain, B. P., Mahoney, M. W., Bronstein, M. M. & Mishra, S. Gradient gating for deep multi-rate learning on graphs. In *International Conference on Learning Representations* (ICLR, 2023).
113. Alon, U. & Yahav, E. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations* (ICLR, 2021).
114. Topping, J., Di Giovanni, F., Chamberlain, B. P., Dong, X. & Bronstein, M. M. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations* (ICLR, 2022).
115. Dimitrov, R., Zhao, Z., Abboud, R. & Ceylan, I. I. PlanE: representation learning over planar graphs. Preprint at https://doi.org/10.48550/arXiv.2307.01180 (2023).
116. Hosseinzadeh, M. M., Cannataro, M., Guzzi, P. H. & Dondi, R. Temporal networks in biology and medicine: a survey on models, algorithms, and tools. *Netw. Model. Anal. Health Inform. Bioinform.* **12**, 10 (2023).
117. Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations* (ICLR, 2017).
Graph convolutional network was the architecture that set off the recent years of development of GNNs.

Acknowledgements
The authors thank R. Wu, S. Yang, D. Lim, A. Corso and M.-M. Troadec for their help in reviewing the manuscript before submission. The authors also thank B. Jing, F. Di Giovanni, J. Yim, C. Vignac and F. Faltings for useful discussions. This work was supported by the NSF Expeditions grant (award 1918839), the Machine Learning for Pharmaceutical Discovery and Synthesis (MLPDS) consortium, the DTRA Discovery of Medical Countermeasures Against New and Emerging (DOMANE) threats program, the DARPA Accelerated Molecular Discovery program, the NSF AI Institute CCF-2112665 and the NSF Award 2134795.

Author contributions
Introduction (R.B., G.C., H.S., S.J. and T.J.); Experimentation (R.B., G.C., H.S., S.J. and T.J.); Results (R.B., G.C., H.S., S.J. and T.J.); Applications (R.B., G.C., H.S., S.J. and T.J.); Reproducibility and data deposition (R.B., G.C., H.S. and S.J.); Limitations and optimizations (R.B., G.C., H.S., S.J. and T.J.); Outlook (R.B., G.C., H.S., S.J. and T.J.); overview of the Primer (all authors).

Competing interests
The authors declare no competing interests.

Additional information
Peer review information *Nature Reviews Methods Primers* thanks Jiliang Tang; Siddhartha Mishra, who co-reviewed with Konstantin Rusch; and Rex Ying, who co-reviewed with Tinglin Huang, for their contribution to the peer review of this work.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Related links
ChEMBL: https://www.ebi.ac.uk/chembl/
Chemprop: https://github.com/chemprop/chemprop
Deep Graph Library: https://www.dgl.ai/
e3nn: https://e3nn.org/
PDBBind: http://www.pdbbind.org.cn/
Protein Data Bank: https://www.rcsb.org/
PyTorch Geometric: https://pytorch-geometric.readthedocs.io/en/latest/